# RESTRICCIONS I INTEGRITAT DE DADES

Professor: Daniel Jesús García Martínez

Versió: febrer 2025

| | | |
|---|---|---|
| iEbre INSTITUT DE L'EBRE TORTOSA | Departament d'Informàtica | DAM MP10 |

**RESTRICCIONS SQL** **3**
RESTRICCIONS SOBRE EL VALOR D'UN CAMP 3
RESTRICCIONS SOBRE LA UNICITAT 4
MODIFICACIONS EN BASE DE DADES 4
**RESTRICCIONS PYTHON** **5**
**CONCLUSIÓ** **6**

# RESTRICCIONS SQL

La forma més directa d'afegir restriccions a un model per tal de garantir que les dades tenen la integritat adequada és definir "constraints" que s'apliquen en l'àmbit de base de dades, és a dir, de SQL. Aquest mètode garanteix que qualsevol canvi en les dades de la taula (per part d'una aplicació o directament sobre la base de dades) serà vàlid a nivell SQL.

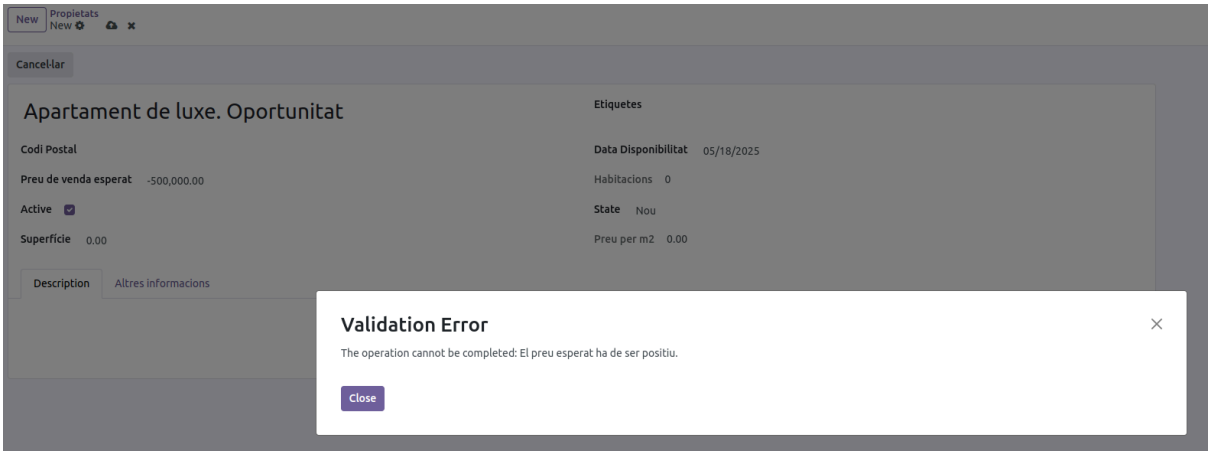
## RESTRICCIONS SOBRE EL VALOR D'UN CAMP

Per afegir una restricció que comprovi el valor a nivell SQL afegim el bloc següent a la definició de la classe:

```
   _sql_constraints = [

       ('check_positive_expected_selling_price',
'CHECK(expected_selling_price >= 0)',
        'El preu esperat ha de ser positiu.')
   ]
```

Aquesta restricció implementarà una "constraint" SQL que limita el valor del camp "expected_selling_price" a un valor no negatiu amb el seu corresponent missatge d'error:

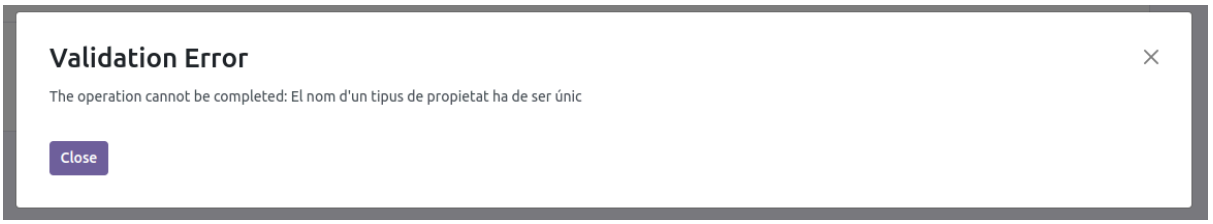


## RESTRICCIONS SOBRE LA UNICITAT

Una restricció habitual és la que permet definir que els valors d'una columna en una taula siguin únics, és a dir, no hi hagi valors repetits. En aquest cas afegiríem la rstricció indicant sobre quina columna s'aplica la clàusula "unique":

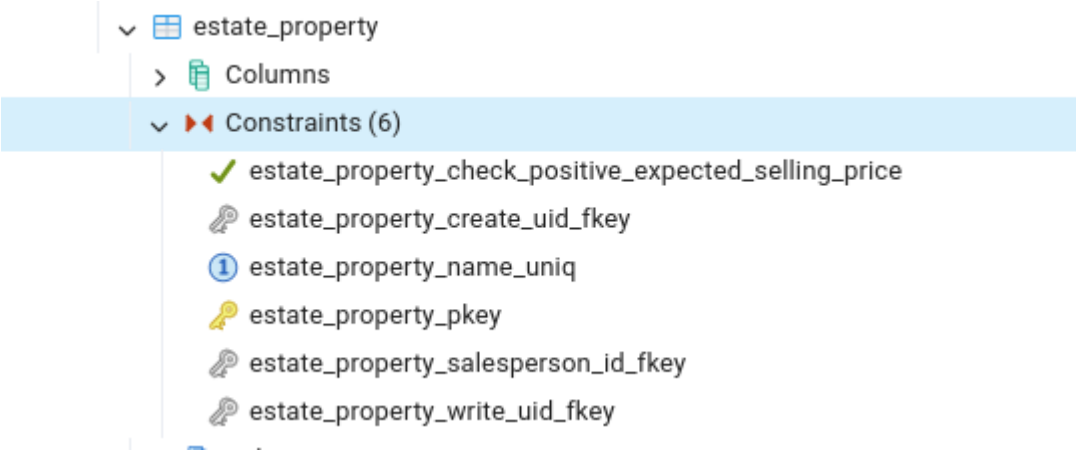
```
    _sql_constraints = [
    ('name_uniq', 'unique(name)', "El nom d'un tipus de propietat ha de ser
únic"),
    ]
```

Si afegim aquesta restricció al nom d'un tipus de propietat i provem de crear un duplicat obtenim el missatge d'error:




## MODIFICACIONS EN BASE DE DADES

A conseqüència de les restriccions creades en el codi de la classe, en la base de dades s'han creat les corresponents "constraints":



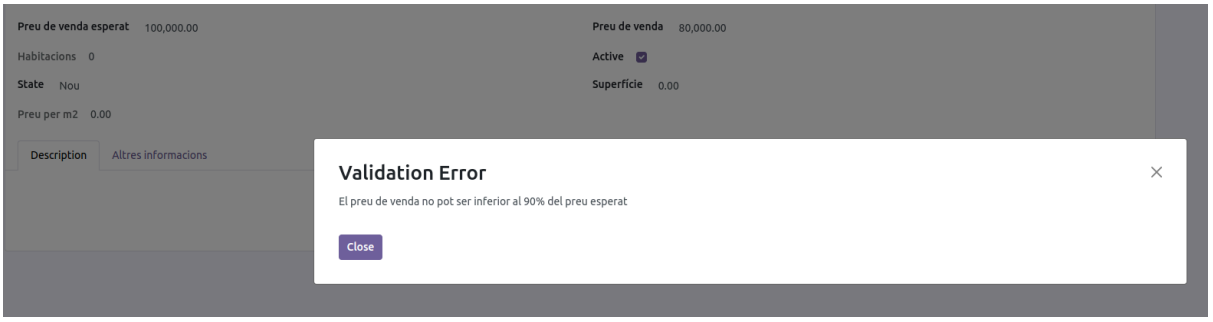
# RESTRICCIONS PYTHON

Si les restriccions a nivell SQL no són suficients i necessitem fer comprovacions més complexes, es pot implementar també la restricció a escala de la lògica de la classe Python.

Definirem un mètode que implementi la restricció indicant amb el decorador api.constrains sobre quins camps depèn la restricció. D'aquesta manera, quan hi hagi un canvi en un dels camps indicats, s'aplicarà la restricció:

```
    @api.constrains('selling_price','expected_selling_price')
    def _check_selling_price_vs_expected(self):
        for record in self:
            if record.selling_price < record.expected_selling_price*0.9:
                raise ValidationError("El preu de venda no pot ser inferior
al 90% del preu esperat")
```

En aquest cas, quan hi ha canvis en selling_price o expected_selling_price, es valida la relació entre els dos camps:



# CONCLUSIÓ

En aquest capítol hem afegit restriccions per validar les dades introduïdes per l'usuari. Aquestes validacions ens permetran assegurar la integritat de les dades enregistrades en la base de dades en funció de la seva naturalesa i de la lògica de negoci de l'aplicació.